28 ENV / ENVIRONMENT VARIABLES — CONFIG THAT LIVES OUTSIDE YOUR CODE

An environment variable is a key/value pair that lives in the OS process running your app, not in a source file. Node exposes all of them on the global `process.env` object — every value on it is a STRING, even if it "looks like" a number or boolean.

This project already uses one, right now, with zero extra packages:

```
// main.ts
await app.listen(process.env.PORT ?? 3000);
console.log(`Server is running on port ${process.env.PORT ?? 3000}`);
```

process.env.PORT is undefined unless something set it before Node started — that's exactly why `?? 3000` is there: a fallback for when the env var was never provided.

WHY BOTHER — THE PROBLEM ENV VARS SOLVE

Without env vars, config (ports, API keys, DB URLs, the `VALID_API_KEY = 'secret123'` in auth.guard.ts) gets hardcoded directly into source files. That's a problem for three reasons:

1. SECRETS IN GIT — a hardcoded API key or DB password gets committed and pushed, permanently, to your repo history.
2. ONE VALUE PER ENVIRONMENT — your laptop, a staging server, and production all need DIFFERENT ports/DB URLs/keys, but it's the SAME source code running in all three.
3. NO REBUILD TO RECONFIGURE — changing a hardcoded value means editing code and redeploying; an env var can change per-deploy with no code change at all.

Env vars fix all three: the VALUE lives outside the code (so it's not committed), and each environment (dev/staging/prod) supplies its own value for the same variable NAME.

HOW A VALUE ACTUALLY GETS INTO process.env

Three common ways, from simplest to most "real project":

1. SET DIRECTLY ON THE COMMAND LINE (works with zero setup)

```
  PORT=4000 node dist/main.js          # bash
  $env:PORT=4000; node dist/main.js    # PowerShell (this project's shell)

Run this project right now and it'll listen on 4000 instead of
3000 — no code change, because main.ts already reads
process.env.PORT.
```

2. A .env FILE + dotenv (the standard local-dev pattern) Instead of typing env vars by hand every time, you put them in a git-ignored `.env` file at the project root:

```
  # .env  (never committed — add it to .gitignore)
  PORT=4000
```

```
  DATABASE_URL=postgres://localhost:5432/mydb
  API_KEY=secret123

and load it at the top of the process, before anything reads
process.env. Plain Node does this with the `dotenv` package:

  import 'dotenv/config'; // must run before other imports use process.env
  console.log(process.env.PORT); // '4000'
```

3. @nestjs/config — THE NEST-IDIOMATIC WAY (not installed in this project yet — this project currently reads process.env directly, which is fine at this scale, but ConfigModule is the standard next step)

```
  npm install @nestjs/config

  // app.module.ts
  import { ConfigModule } from '@nestjs/config';

  @Module({
    imports: [
      ConfigModule.forRoot({ isGlobal: true }), // reads .env automatically, no manual dotenv import
needed
      EmployeeModule, CategoryModule, UserModule, TaskModule,
    ],
    // ...
  })
  export class AppModule {}

Then INJECT config instead of reaching for process.env everywhere:

  import { ConfigService } from '@nestjs/config';

  @Injectable()
  export class AuthGuard implements CanActivate {
    constructor(private config: ConfigService) {}

    canActivate(context: ExecutionContext): boolean {
      const validKey = this.config.get<string>('API_KEY'); // instead of a hardcoded VALID_API_KEY
      const request = context.switchToHttp().getRequest<Request>();
      return request.headers['x-api-key'] === validKey;
    }
  }
```

This is strictly better than raw process.env once a project grows:

```
ConfigService is INJECTABLE (mockable in tests, unlike a global),
supports typed access, validation schemas (via Joi/class-validator),
namespacing, and loading different files per environment
(`.env.development`, `.env.production`) automatically.
```

EXAMPLE FROM THIS PROJECT — WHERE process.env SHOULD REPLACE A HARDCODE

auth.guard.ts currently has this, with its own TODO admitting it:

```
const VALID_API_KEY = 'secret123'; // TODO: move to env/ConfigService in a real app
```

That's the textbook case env vars exist for: a secret, hardcoded, that differs between "my machine" and "production," committed in plain text right now. The fix, even without installing @nestjs/config:

```
const VALID_API_KEY = process.env.API_KEY ?? 'secret123'; // real key in prod, fallback for local dev
```

...paired with a `.env` file (git-ignored) providing the real value locally, and the deployment platform (Docker, a host's dashboard, CI secrets) providing it in every other environment.

⚠ TYPE GOTCHA — EVERYTHING ON process.env IS A STRING

PORT=4000	-→	process.env.PORT === '4000'	(a string, not 4000)
DEBUG=true	-→	process.env.DEBUG === 'true'	(a string, not the boolean true)

```
if (process.env.DEBUG) { ... }        // truthy for BOTH 'true' AND 'false' — both are non-empty strings!
if (process.env.DEBUG === 'true') { ... }  // correct way to check a boolean-flavored env var
```

app.listen(process.env.PORT ?? 3000) works here specifically because Node's http server accepts a numeric STRING port just fine — but the moment you need a real number (e.g. `id + 1`), you must convert it yourself: `Number(process.env.PORT)`.

★ RULE OF THUMB

Anything that (a) is a secret, (b) differs between your machine and someone else's, or (c) might need to change without a redeploy — belongs in an env var, not a hardcoded constant. For a project this size, reading process.env directly (as main.ts already does for PORT) is perfectly fine; reach for @nestjs/config's ConfigModule/ConfigService once you have more than a couple of values, want validation, or want them injectable for testing.

See also: **theory/27-lifecycle-events.txt**, **theory/17-guards.txt** (auth.guard.ts's VALID_API_KEY is the concrete example above), **theory/07-dependency-injection.txt**